You said:

user -> CDN1, CDN2,.....,CDN N -> app server
why is not good idea ?

ChatGPT said:

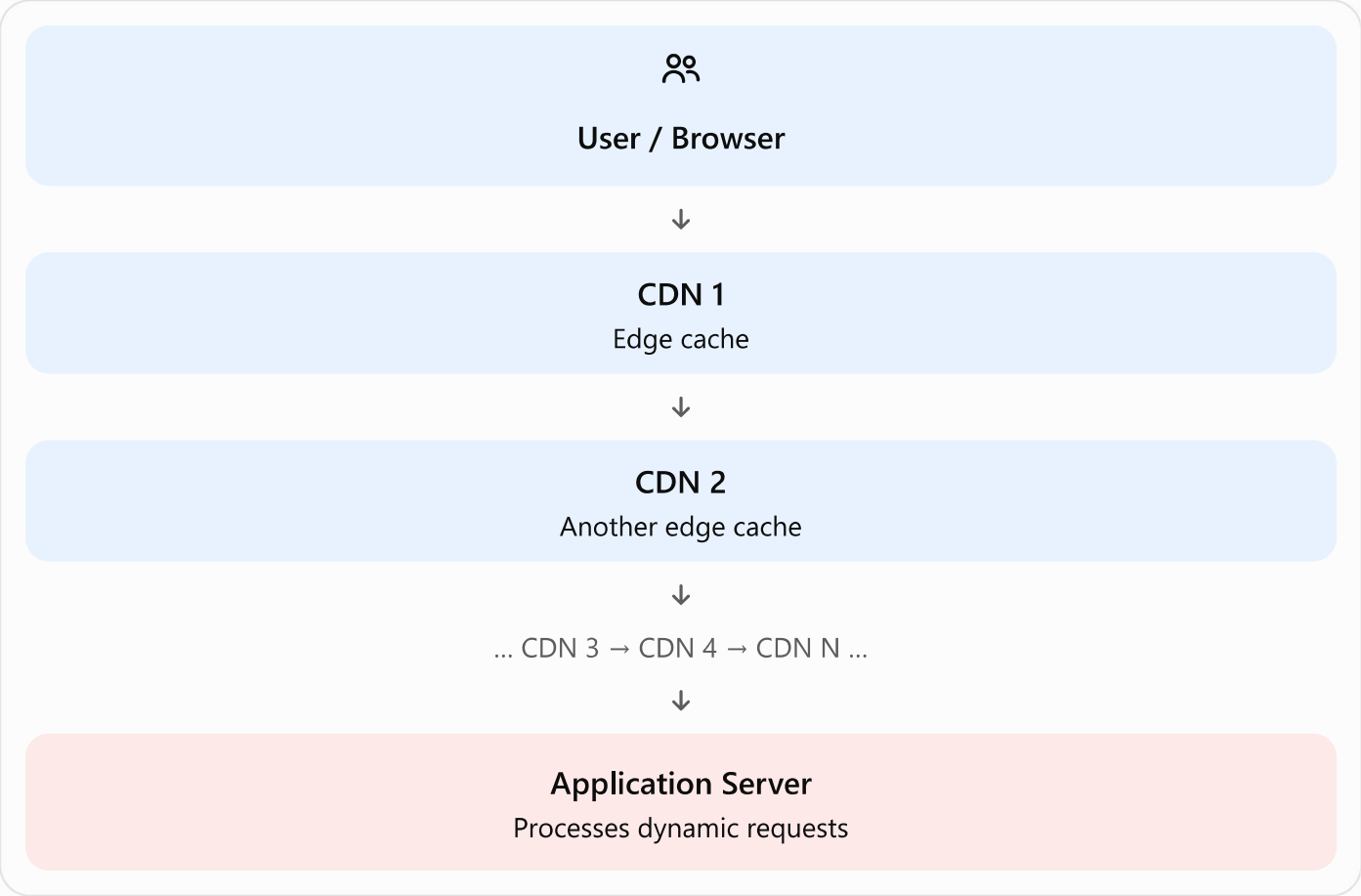
Your architecture is:

`User → CDN 1 → CDN 2 → … → CDN N → Application Server`

The main problem is that chaining multiple CDNs in a single request path usually adds latency, cost, and operational complexity without providing proportional benefits.

Let's understand this using a real-world example.

1. Your proposed architecture



Imagine a user requests `https://example.com/products`.

If the requested content is not cached at CDN 1, the request may need to travel to CDN 2. If it misses there, it continues through the chain until it reaches the origin application server.

The response then travels back through the chain.

2. Why this is usually a bad idea

1. Increased latency

Each additional CDN can introduce another network hop, TLS handling, routing, and cache lookup. On a cache miss, the cumulative delay can increase response time.

2. Higher infrastructure cost

Multiple CDN providers or layers can introduce additional request charges, origin-transfer charges, and operational costs. The same request may be processed by several layers.

3. Cache duplication and invalidation complexity

Several CDNs may cache the same image, JavaScript bundle, or HTML response. When content changes, you may need to manage TTLs and invalidation across multiple layers.

4. More failure points

If one CDN becomes unavailable or misconfigured, the entire request path can fail unless you have an appropriate bypass or failover strategy.

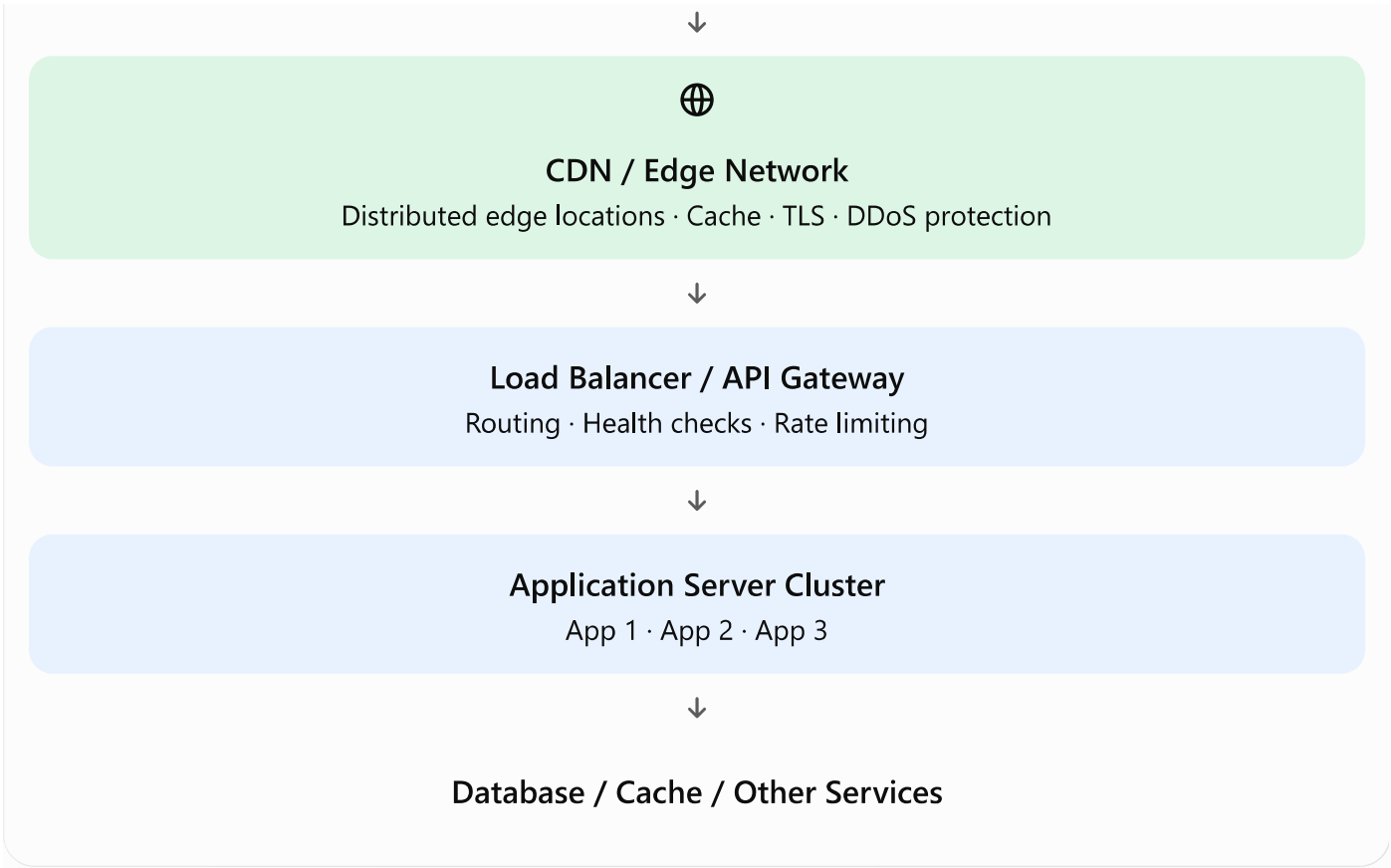
5. Harder troubleshooting

You must trace cache hits, cache misses, HTTP headers, TLS certificates, rate limits, and security rules across several systems.

3. Recommended architecture

For most web applications, use **one CDN layer in front of your application infrastructure**.





How it works

1. A user requests a resource.
2. The CDN checks whether the resource is cached at an edge location.
3. **Cache hit:** the CDN returns the content without contacting your application server.
4. **Cache miss:** the CDN fetches the content from the origin and may cache it according to your rules.
5. Dynamic requests are forwarded to your application infrastructure.

For example, product images and versioned JavaScript files are often good CDN cache candidates. Personalized banking balances generally should not be publicly cached.

4. But are multiple CDNs always bad?

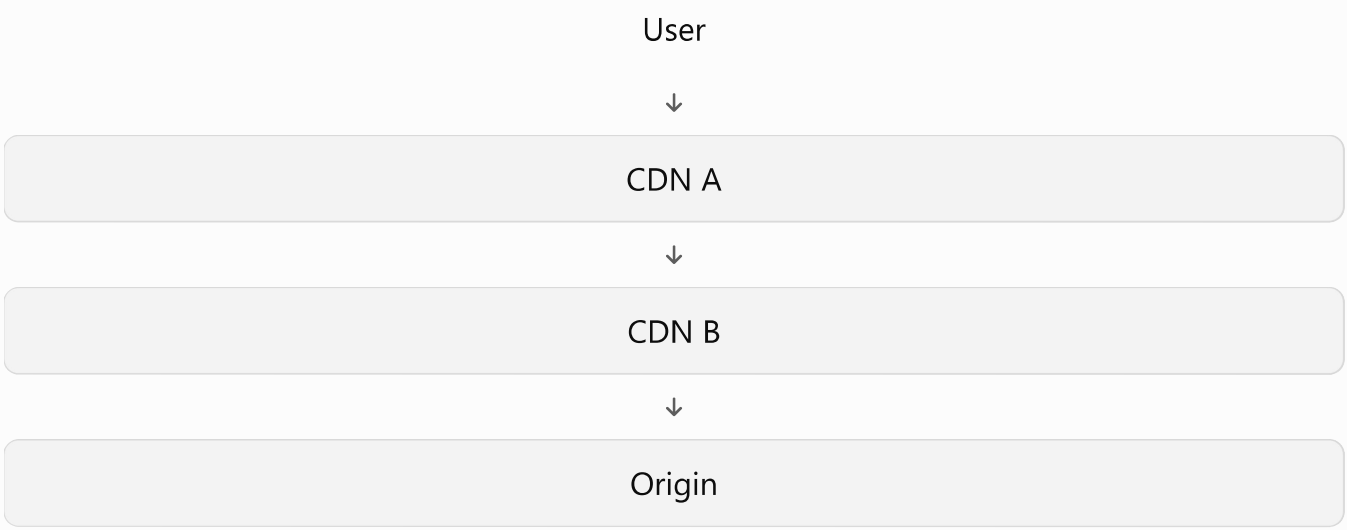
No. There are legitimate architectures that use multiple CDNs.

Architecture	Typical purpose
CDN 1 → CDN 2	CDN layering, such as a regional CDN in front of an origin-facing CDN
CDN A or CDN B	Multi-CDN routing for availability and performance

CDN → origin shield → app server	Reduce repeated origin requests and protect the origin
CDN → load balancer → app servers	Common architecture for scalable web applications

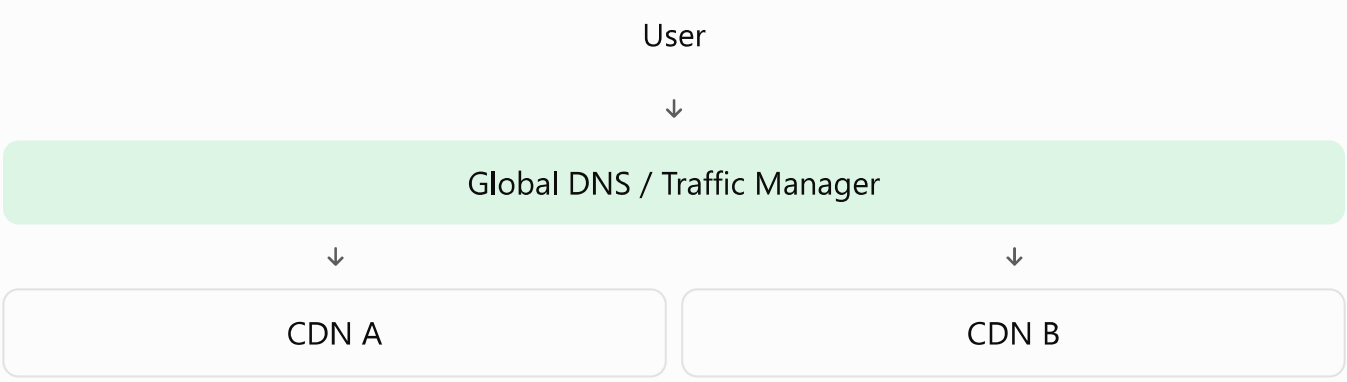
The important distinction is **serial chaining versus parallel choice**.

A. Serial chaining



Each request traverses the configured chain. Useful only when each layer has a clear purpose.

B. Multi-CDN with routing



Traffic is directed to an appropriate CDN based on performance, availability, geography, or policy. Both CDNs may use the same origin.

Interview-ready answer: Putting CDN 1, CDN 2, … CDN N in a serial request path is generally inefficient because it adds network hops, cost, cache-management complexity, and potential failure points. A single well-configured CDN is sufficient for many applications. When higher

resilience or geographic performance is required, a multi-CDN architecture with intelligent traffic steering is often more appropriate.

You said:

Chaining multiple Content Delivery Networks (CDNs) sequentially—where one CDN forwards requests to another CDN before finally reaching the origin application server—is generally a bad architecture because **it multiplies latency, dramatically increases operational complexity, and drives up financial costs without providing extra benefits.**

While multi-CDN architectures are common, they are deployed in *parallel* (switching between CDNs based on user location or performance), not in a serial chain.

Here is why chaining CDNs sequentially is a poor design choice:

1. Increased Latency (The Opposite of a CDN's Purpose)

CDNs are designed to minimize latency by serving content close to the user. When you chain them sequentially, you introduce severe performance bottlenecks:

- **Multiple DNS Lookups:** The request must resolve hostnames for CDN 1, then CDN 2, and so on, adding significant overhead.
- **Extra TCP/TLS Handshakes:** Each hop requires establishing new secure connections, drastically slowing down the Time to First Byte (TTFB).
- **Longer Request Paths:** If CDN 1 does not have the asset cached, it queries CDN 2. If CDN 2 misses, it queries the next. This serialized execution makes cache misses devastatingly slow.

2. Cache Invalidation Nightmares

Managing cache consistency becomes nearly impossible.

- **Stale Content:** If you update an image or a webpage, you must purge the cache across every single CDN in the chain.
- **Synchronization Failures:** If a purge command fails or gets delayed at CDN 2, CDN 1 will continue to fetch and cache the old content from CDN 2, permanently serving stale data to your users until the Time-to-Live (TTL) naturally expires.

3. Compounded Points of Failure

In a chain, **your availability is only as strong as the weakest link.** If any single CDN in the middle of the chain experiences an outage, undergoes misconfiguration, or suffers a routing issue, your entire application goes down for the user.

4. Exploding Costs

Most CDNs charge for data egress (bandwidth out). In a chained setup:

- The application server pays to send data to CDN N.
- CDN N charges egress to send it to CDN N-1.
- This repeats until CDN 1 sends it to the user.
  You end up **paying multiple times for the exact same byte of data** to traverse the network.

5. Stripped Header and Security Context

As requests pass through multiple proxy layers, critical client information often gets lost or malformed:

- **IP Spoofing & Tracking:** Managing headers like `X-Forwarded-For` becomes messy, making it difficult for your application server to know the user's actual geographic location or true IP address.
- **WAF Conflicts:** If multiple CDNs have Web Application Firewalls (WAF) enabled, a legitimate request modified by CDN 1's optimization features might be flagged as a security threat and blocked by CDN 2.

The Better Alternative: Parallel Multi-CDN

If you want the redundancy and geographic coverage of multiple CDNs, deploy them in **parallel**, not in a series.

Use a smart **DNS router or Traffic Manager** (like Route 53, NS1, or Cedexis) at the very front. The router evaluates the user's location and the real-time health of the CDNs, then routes the user directly to the single best CDN, which connects straight to your application server.

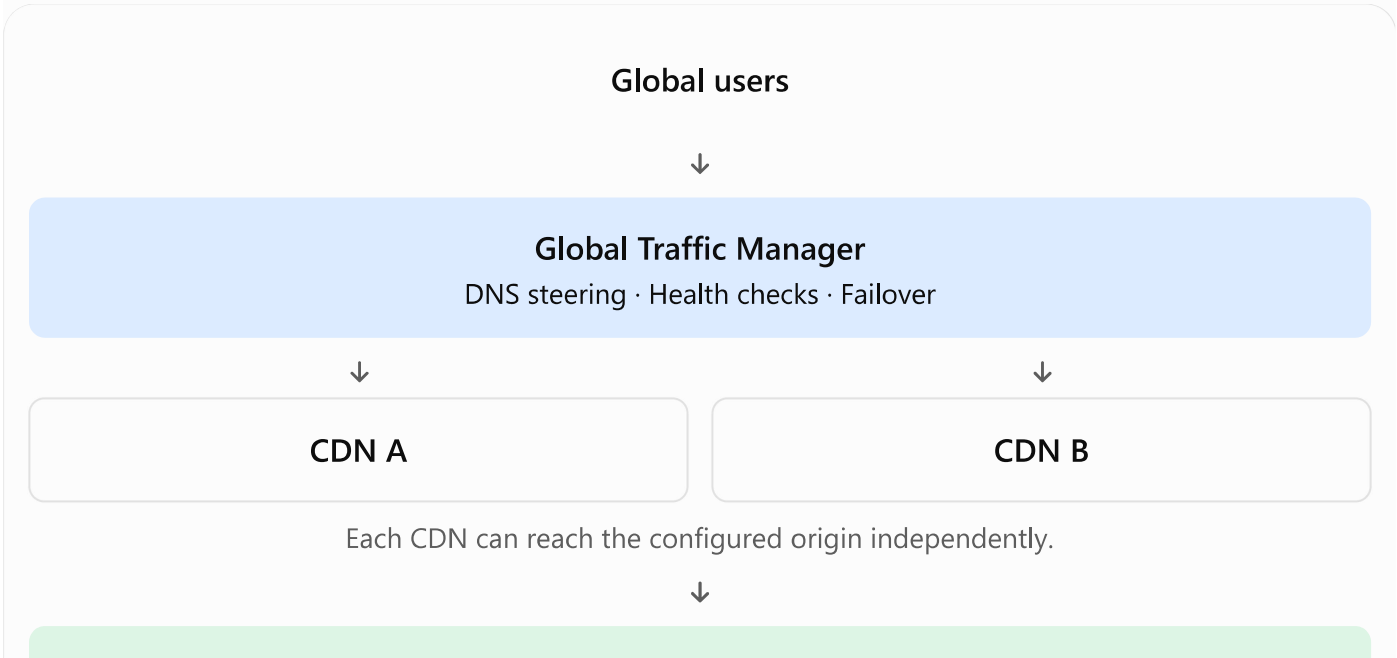
ChatGPT said:

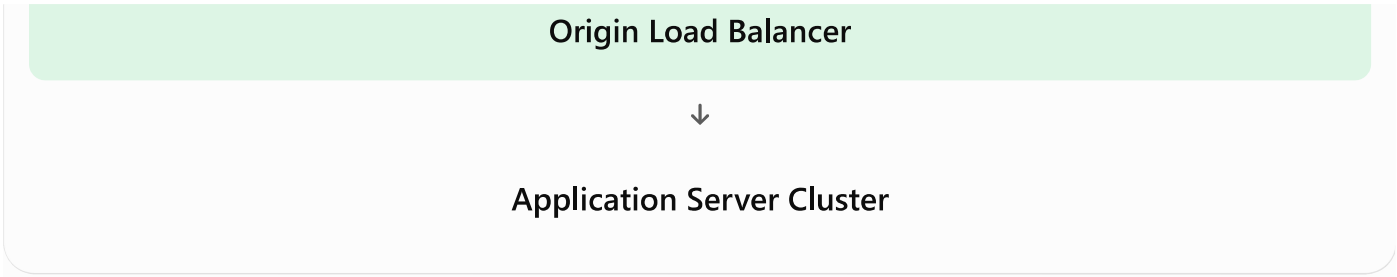
Your explanation is strong and largely correct. I would make a few technical refinements to make it **production-grade and interview-ready**.

Key technical corrections

Your statement	More precise explanation
Multiple DNS lookups	Not necessarily. CDN-to-CDN origin requests may reuse resolved DNS records and connections.
Extra TCP/TLS handshakes	Possible, but not for every request. Persistent connections and HTTP/2 or HTTP/3 can reduce overhead.
Cache invalidation becomes nearly impossible	It becomes more complex, but coordinated purges, versioned URLs, and TTLs can manage it.
One CDN outage brings everything down	True for a strict chain without bypass or failover. Redundancy can mitigate this.
Paying multiple times for every byte	Potentially, but depends on CDN contracts, caching behavior, origin configuration, and egress pricing.
Multi-CDN should always be parallel	Parallel selection is common for resilience, but serial CDN layering can be justified for specific requirements.

Recommended production architecture





One important detail: DNS-based steering usually influences which CDN a user connects to; it does not necessarily make a new decision for every individual request. Health checks, DNS caching, session behavior, and CDN-specific routing affect failover.

Final takeaway: Avoid serial CDN chaining unless each layer provides a deliberate architectural benefit. For multi-CDN resilience, use traffic steering, independent CDN paths, a shared or appropriately replicated origin, and tested failover procedures.

You said:

when we can have multiple load balancer then why not CDN?

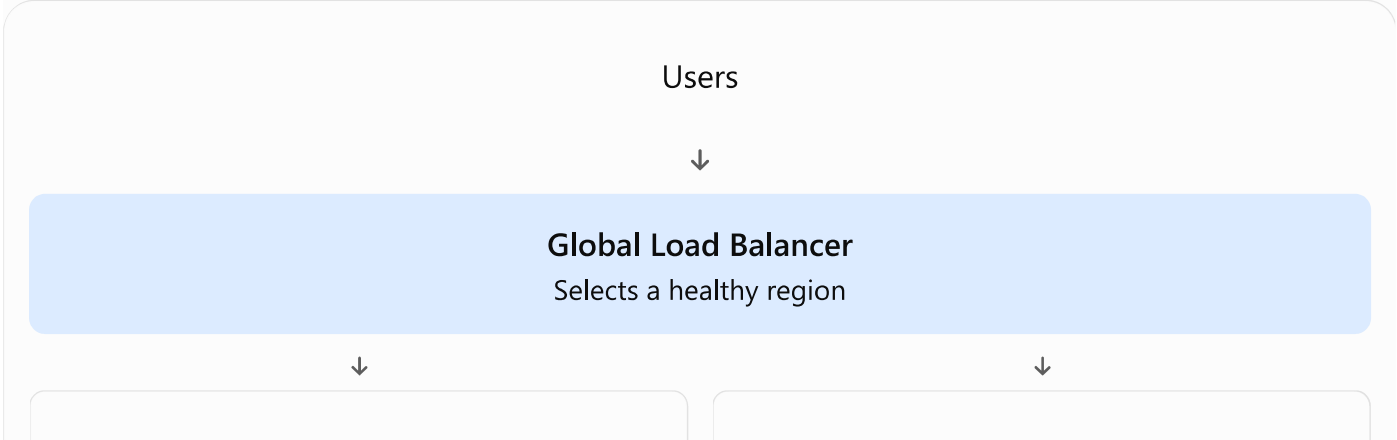
ChatGPT said:

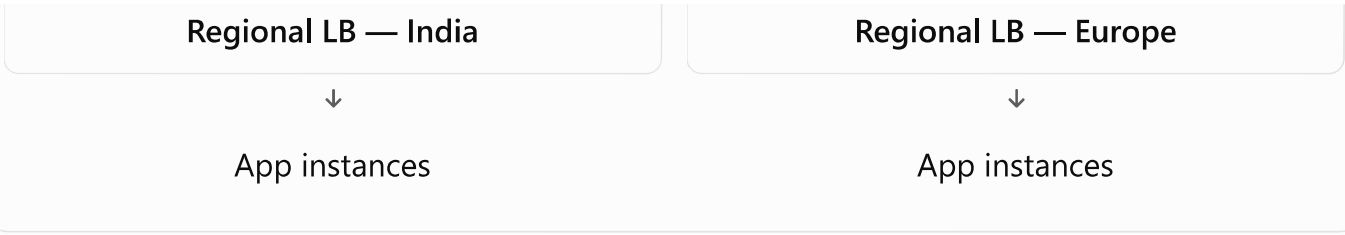
Excellent question. The key is that **we can absolutely have multiple CDNs, just as we can have multiple load balancers. The important difference is how they are arranged and what problem each layer solves.**

Multiple load balancers are common in production, and multiple CDNs are common too. Neither is inherently a bad design.

1. Why multiple load balancers work

Consider a large banking application.





Here, the two load-balancing layers perform different jobs:

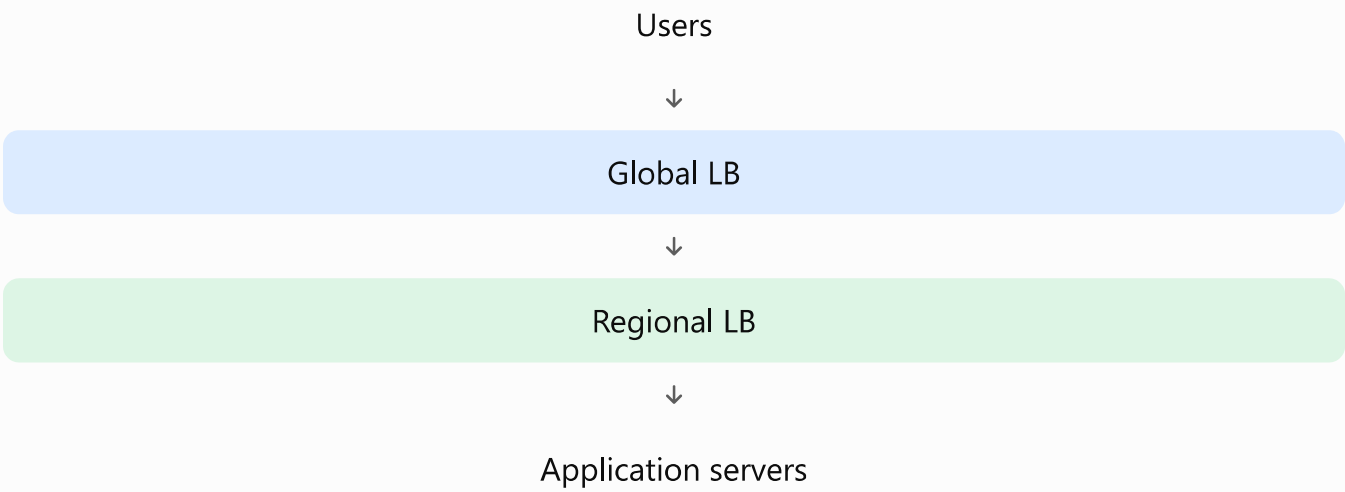
- **Global load balancer:** chooses a region based on health, geography, latency, or routing policy.
- **Regional load balancer:** distributes requests across application instances within that region.

Each layer solves a distinct scaling or availability problem.

2. Why can't we use multiple CDNs in the same way?

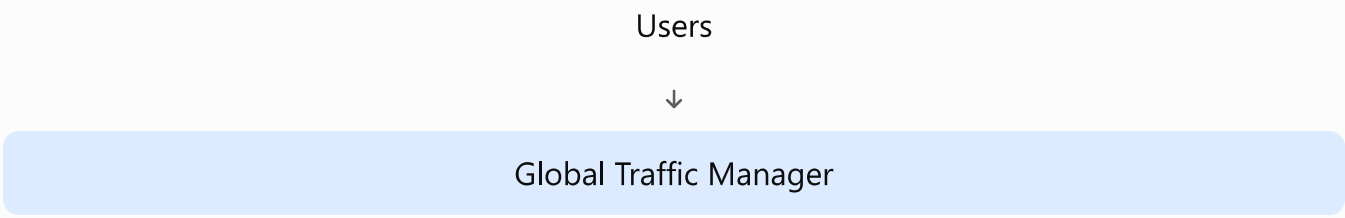
We can! Let's compare the architectures.

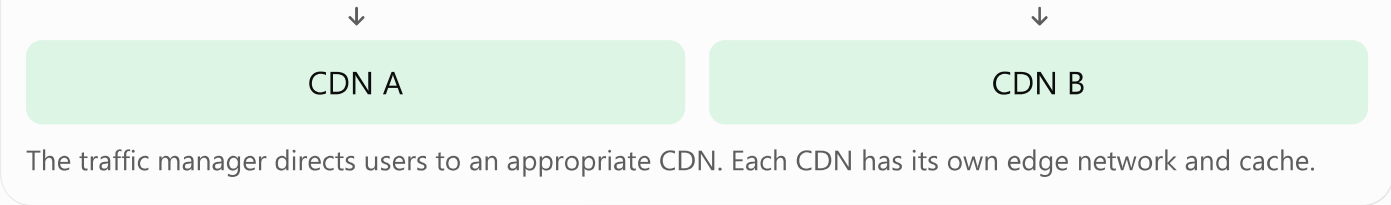
A. Multiple load balancers in a hierarchy



Each layer distributes traffic at a different scope.

B. Multiple CDNs in parallel



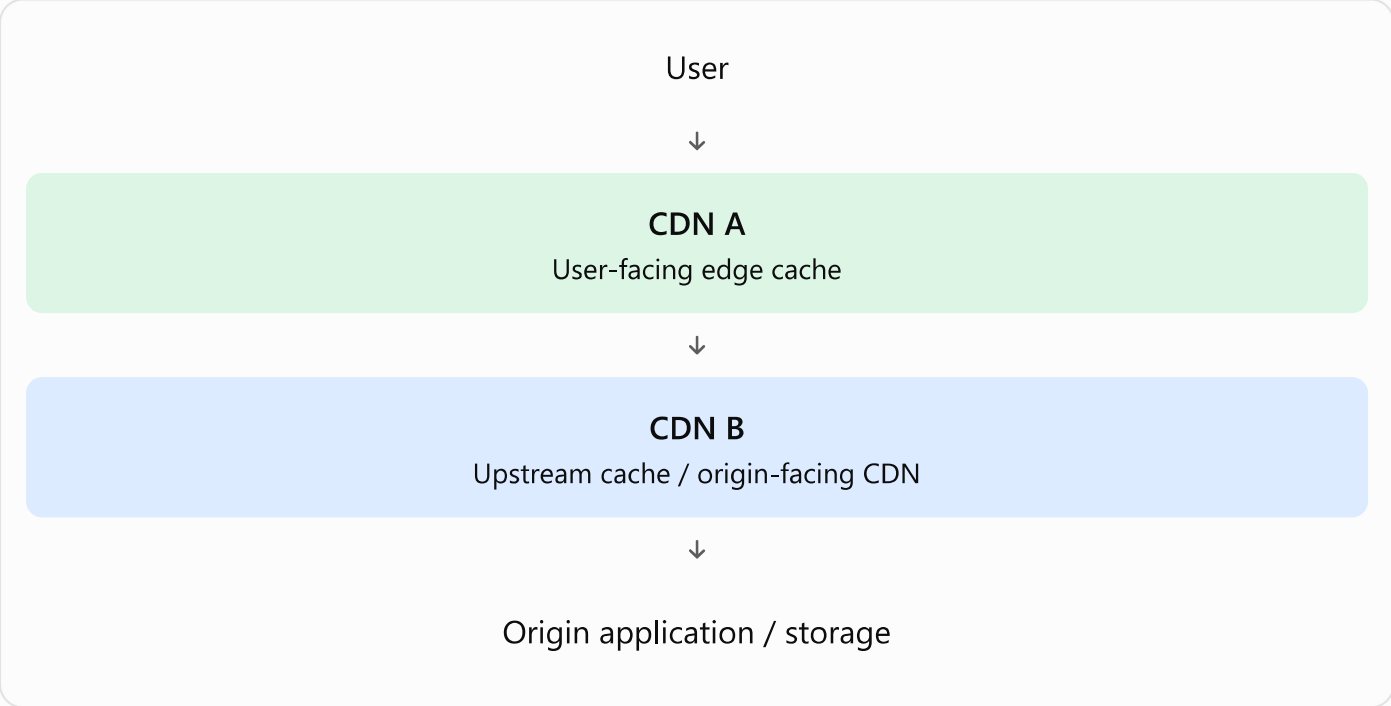


The traffic manager directs users to an appropriate CDN. Each CDN has its own edge network and cache.

Both architectures are valid. The difference is that **load balancers distribute requests among destinations, whereas CDNs primarily serve cached content from geographically distributed edge locations**.

3. What about chaining CDN A → CDN B?

This is also possible.



This can make sense if CDN A serves a specialized user-facing role and CDN B reduces origin load or provides a separate distribution capability.

However, if both CDNs do essentially the same work, chaining them may add cost, latency, cache invalidation challenges, and failure dependencies without enough benefit.

4. The key comparison

Question	Multiple load balancers	Multiple CDNs
Is it valid?	Yes	Yes

Common arrangement	Hierarchical or regional	Parallel, sometimes layered
Main purpose	Distribute traffic and improve availability	Cache and deliver content close to users
Does each layer add value?	Ideally, each has a distinct routing scope	Ideally, each has a distinct caching or distribution role
Main risk of unnecessary layers	Extra hops and complexity	Extra hops, duplicated caching, cost and complexity

Interview-ready answer

"Multiple load balancers and multiple CDNs are both valid architectural choices. We use multiple load balancers when different layers distribute traffic at different scopes, such as global-to-regional-to-instance. We can also use multiple CDNs for provider redundancy, geographic coverage, or layered caching. The problem is not the number of layers; it is adding layers without a clear purpose. If multiple CDNs are chained sequentially, we must justify the additional latency, cache coordination, cost, and failure dependencies."

Remember: Don't ask *"Can we add another CDN?"* Ask *"What additional capability does this CDN provide that the existing architecture does not?"*

You said:

in production, we usually keep one CDN server right ?

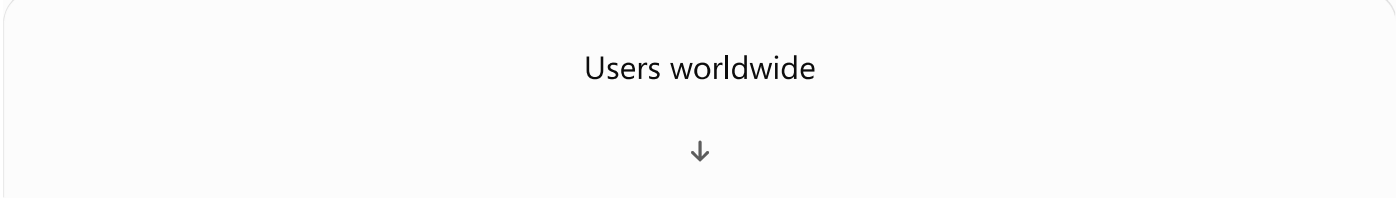
ChatGPT said:

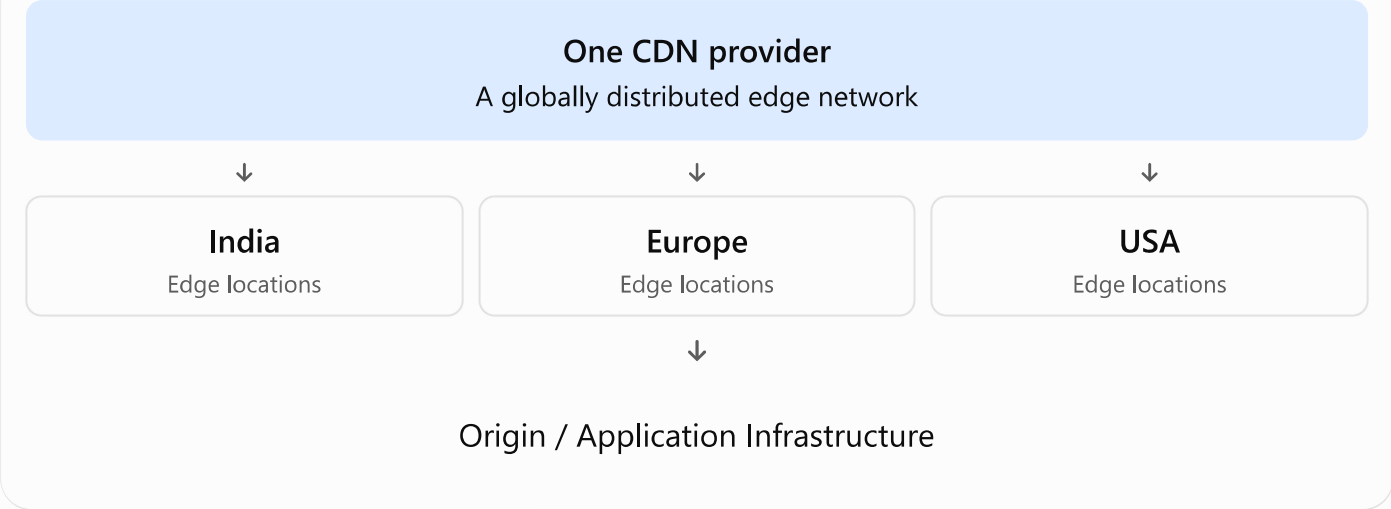
Yes — **in many production applications, we use one CDN provider or one CDN layer**, but that does **not** mean there is only one physical CDN server.

This distinction is important in system design.

1. One CDN provider ≠ one CDN server

For example, your application uses Cloudflare or Amazon CloudFront.





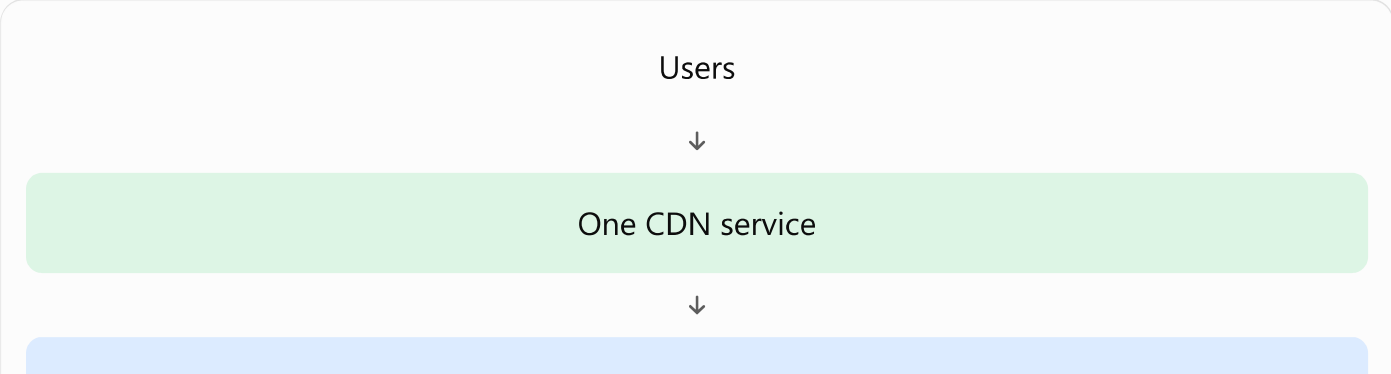
Your application typically configures one CDN service, while the CDN provider operates many geographically distributed edge locations and supporting infrastructure.

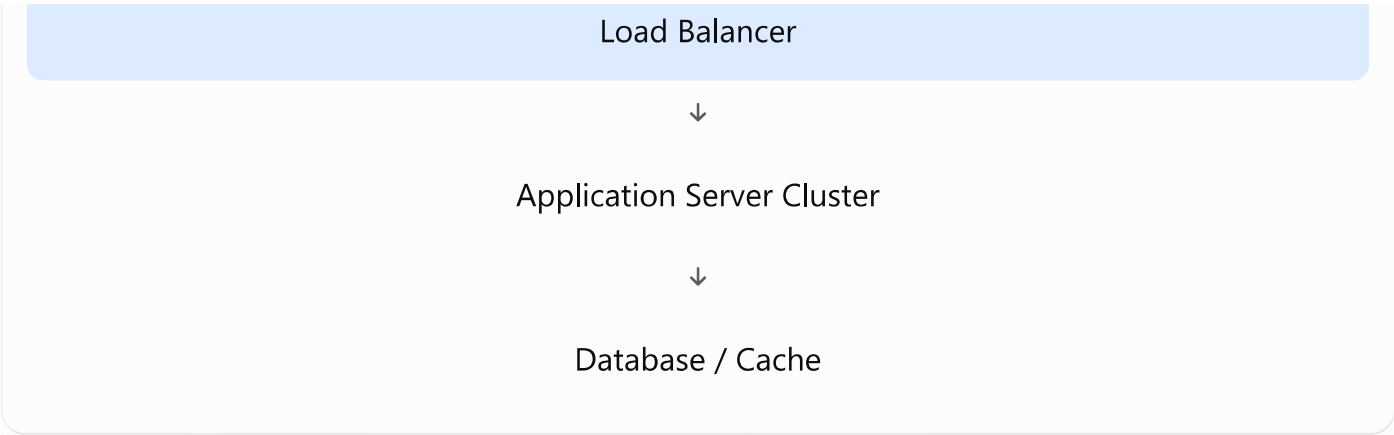
You do not normally deploy and manage each edge server yourself.

2. What is common in production?

Architecture	Typical use
One CDN provider with many edge locations	Common for most applications
One CDN + load balancer + multiple app servers	Common production architecture
Two CDN providers with traffic steering	Large-scale systems requiring additional resilience or performance options
CDN A → CDN B → CDN C	Possible, but generally requires a specific architectural justification

3. A typical production architecture





Interview takeaway: In production, we commonly use one CDN provider or CDN layer backed by a distributed edge network. We scale and provide redundancy within that service, and introduce a second CDN provider when the business requirements justify multi-CDN routing or failover.